Laporan Praktikum Algoritma & Pemrograman

Semester Genap 2025/2026

SAYA MENYATAKAN BAHWA LAPORAN PRAKTIKUM INI SAYA BUAT DENGAN USAHA SENDIRI TANPA MENGGUNAKAN BANTUAN ORANG LAIN. SEMUA MATERI YANG SAYA AMBIL DARI SUMBER LAIN SUDAH SAYA CANTUMKAN SUMBERNYA DAN TELAH SAYA TULIS ULANG DENGAN BAHASA SAYA SENDIRI.

SAYA SANGGUP MENERIMA SANKSI JIKA MELAKUKAN KEGIATAN PLAGIASI, TERMASUK SANKSI TIDAK LULUS MATA KULIAH INI.

NIM	**71251172**
Nama Lengkap	**Ang, Joshua Alexander Hartono**
Minggu ke / Materi	**10 / Manajemen File**

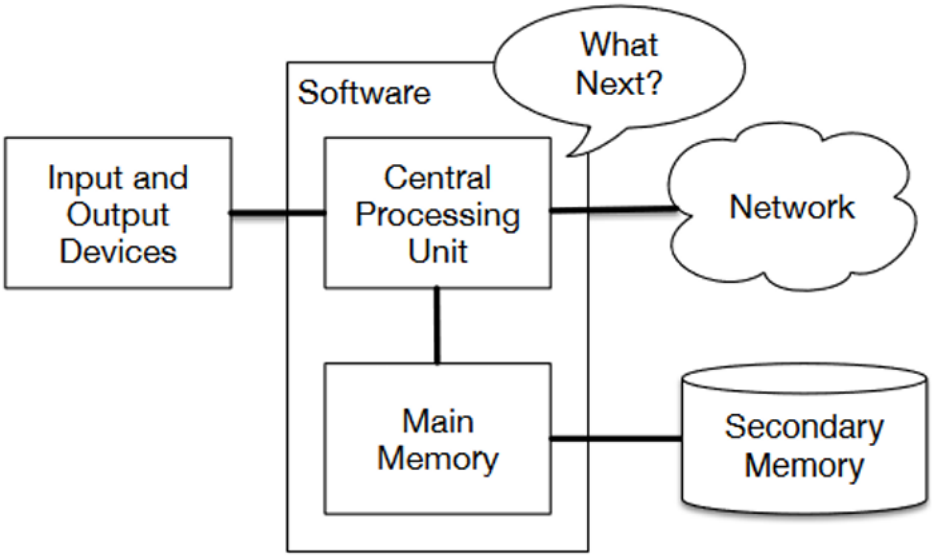
PROGRAM STUDI INFORMATIKA
FAKULTAS TEKNOLOGI INFORMASI
UNIVERSITAS KRISTEN DUTA WACANA
YOGYAKARTA
2026

BAGIAN 1: MATERI MINGGU INI (40%)

Pengantar File

Program yang berjalan di komputer membutuhkan memory primer untuk menyimpan data sementara proses berlangsung. Data yang disimpan pada memory ini hanya bersifat sementara dan akan hilan ketiak program selesai dijalankan atau komputer dimatikan. Hal ini disebabkan memory primer memiliki sifat volatile, yaitu tidak mampu menyimpan data secara permanen. Oleh karena itu, program yang menggunakan memory primer tidak akan dapat menyimpan data setelah program dimatikan.

Untuk mengatasi masalah tersebut, digunakanlah secondary memory sebagai media dari penyimpanan tetap. Secondary memory berfungsi iuntuk menyimpan data secara permanen sehingga data tetap ada walaupun komputer dimatikan. Contoh dari secondary memory diantaranya adalah SSD dan harddisk.



Gambar 9.1: Penggunaan memory di komputer. *Sumber:* https://shorturl.at/EEict

File merupakan salah satu bentuk penyimpanan data yang berada di dalam secondary memory. File berisi kumpulan data dalam bentuk bit-bit yang disusun dan disimpan secara teratur sehingga dapat dibaca dan digunakan oleh program. File dapat berupa berbagai jenis, seperti file sistem, file program (binary), file teks, maupun file multimedia. Setiap file menyimpan informasi yang

saling berkaitan dan dianggap sebagai satu kesatuan. Dengan adanya file, data dari program tidak akan hilang walaupun komputer dimatikan. Setiap file memiliki beberapa properti yang menjelaskan karakteristiknya. Properti tersebut meliputi nama file, ukuran, letak di harddisk, owner, hak akses, tanggal akses, dan lain-lain. Informasi ini penting untuk mengatur dan mengelola file di dalam sistem komputer.

Pengaksesan File

Untuk dapat mengakses sebuah file di dalam program, terdapa beberapa langkah yang harus dilakukan secara berurutan. Pertama, pengguna harus menyiapkan nama file beserta path atau lokasi file berada. Setelah itu, buka file agar program dapat berinteraksi dengan file tersebut. Langkah berikutnya adalah melakukan operasi pada file, seperti membaca isi file (read) atau menuliskan data ke file (write). Terakhir, file darus di tutup.

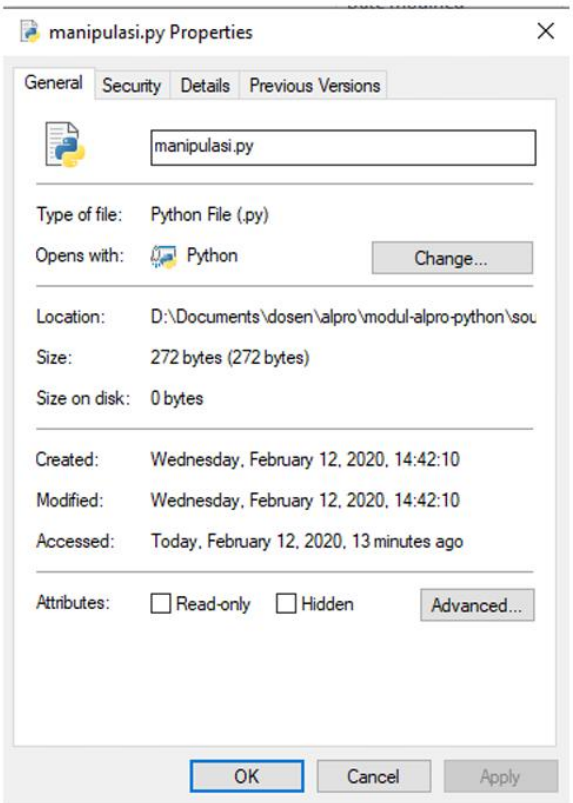
Dalam bahasa Python, file dapat diakses menggunakan fungsi open(). Ketika file dibuka, program akan menghasilkan sebuah handla file yang digunakan sebagai penghubung antara program dan file tersebut. Handle ini berisi informasi seperti nama file, mode akses, dan encoding yang digunakan.

```
handle =  open('mbox-short.txt')
print(handle)
```

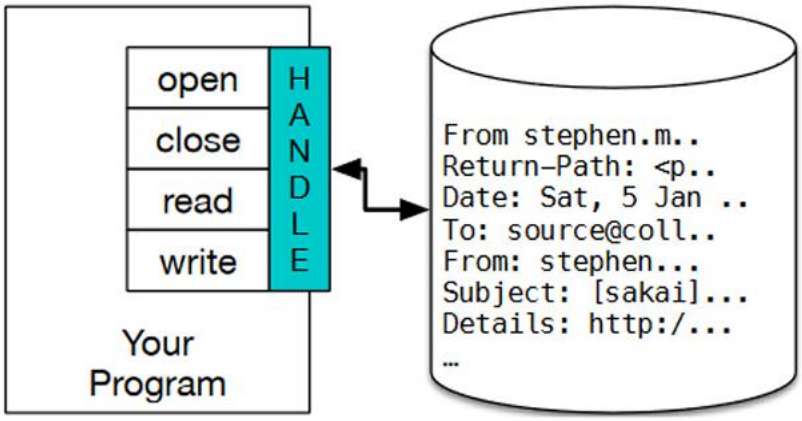
Hasil adalah:

```
<_io.TextIOWrapper name='mbox-short.txt' mode='r' encoding='UTF-8'>
```

Gambar 9.2: Penggunaan fungsi open() untuk membuka file “mbox-short.txt”. *Sumber:* https://shorturl.at/EEict



Gambar 9.3: Contoh property file. *Sumber:* https://shorturl.at/EEict



Gambar 9.4: Ilustrasi handle file. *Sumber:* https://shorturl.at/EEict

Pada file “mbox-short.txt”, hasil yang ditampilkan adalah file dibuka dalam mode read dan dengan encoding UTF-8. Namun, jika file tidak ditemukan maka output akan error. Error yang

muncul biasanya adalah FileNotFoundError yang menunjukkan bahwa file tidak ada dalam lokasi yang ditentukan. Contoh:

```
Traceback (most recent call last):  File "main.py", line 1, in <module>
handle =  open('tidak-ada.txt')
FileNotFoundError: [Errno 2] No such file or directory: 'tidak-ada.txt'
```

Gambar 9.5: Contoh error yang terjadi saat proses handle dan file tidak ditemukan. *Sumber:* https://shorturl.at/EEict

Pada file teks, isi file biasanya terdiri dari beberapa jenis string yang tersusun secara berurutan. Proses pembacaan file teks umumnya dilakukan baris demi baris hingga mencapai akhir dari file yang disebut EOF (End of File).

```
From stephen.marquard@uct.ac.za Sat Jan  5 09:14:16 2008
Return-Path: <postmaster@collab.sakaiproject.org>
Date: Sat, 5 Jan 2008 09:12:18 -0500
To: source@collab.sakaiproject.org
From: stephen.marquard@uct.ac.za
Subject: [sakai] svn commit: r39772 - content/branches/
Details: http://source.sakaiproject.org/viewsvn/?view=rev&rev=39772
...
```

Gambar 9.6: Contoh file teks dan barisnya. *Sumber:* https://shorturl.at/EEict

Manipulasi File

Manipulasi file dalam Python dimulai dengan proses membaca file terlebih dahulu. Langkah selanjutnya adalah membuka file dengan open(), membaca isi file menggunakan perulangan, dan menutup file setelah selesai.

```
handle =  open('mbox-short.txt')
count = 0
for line in handle:
    count = count + 1
print('Line Count:', count)
```

Gambar 9.7: Contoh pembacaan file menggunakan looping. *Sumber:* https://shorturl.at/EEict

Pada program, file dibuka lalu setipa baris dihitung menggunakan variabel count hingga seluruh file selesai dibaca. Hasil akhirnya menunjukkan bahwa jumlah total baris pada line tersebut

adalah 1910 baris. Selain cara tersebut, file juga bisa dibaca sekaligus menggunakan fungsi read(). Dengan cara ini, seluruh isi file disimpan dalam satu variabel string sehingga bisa diproses lebih lanjut. Namun, metode ini cukup boros secara memori jika file berukuran besar. Oleh karena itu, penggunaan read() sebaiknya hanya untuk file yang tidak terlalu besar agar program tetap berjalan lancar.

```
handle =  open('mbox-short.txt')
hasil = handle.read()
print("Ukuran: " + len(hasil) + "bytes")
print("Huruf dari belakang sendiri mundur 16 huruf adalah: " + hasil[-16::1])
```

Gambar 9.8: Contoh pembacaan file menggunakan read(). *Sumber:* https://shorturl.at/EEict

Penyimpanan File

Penulisan file dalam Python dilakukan dengan cara yang hampir sama seperti membaca file, yaitu dengan cara menggunakan fungsi open(), tetapi dengan metode yang berbeda. Jika sebelumnya menggunakan mode r untuk membaca, maka untuk menulis menggunakan mode w. Mode ini akan membuat file baru atau menimpa isi file lama jika file tersebut sudah ada. Setelah file dibuka, program akan mendapatkan handle yang digunakan untuk menulis data ke dalam bentuk file.Untuk menuliskan data ke file, digunakan fungsi write() yang berisi string yang ingin disimpan. Kemudian diakhiri dengan close() untuk menutup file.

```
handle = open('output.txt','w')
tulisan = "teks ini akan dituliskan ke file\n"
handle.write(tulisan)
handle.close()
```

Gambar 9.9: Contoh penyimpanan file. *Sumber:* https://shorturl.at/EEict

BAGIAN 2: LATIHAN MANDIRI (60%)

Link github : https://github.com/angjoshua-sudo/Prak-Alpro-2026.git

SOAL 1

Source code:

minggu_10 > soal 1.py > ...

```python
file1= input("masukkan nama file ke-1: ")
file2= input("masukkan nama file ke-2: ")
isi1= []
isi2= []
count = 0

try:
    handle1 = open(file1)
    handle2 = open(file2)

    for line in handle1:
        line = line.strip()
        isi1.append(line)

    for line in handle2:
        line = line.strip()
        isi2.append(line)

    len1 = len(isi1)
    len2 = len(isi2)

    if len1  > len2:
        panjang = len2
    elif len1 < len2:
        panjang = len1
    else:
        panjang = len1

    for i in range (panjang):
        if isi1[i] != isi2[i]:
            count += 1
            print(f'baris ke-{i+1} berbeda, file1: {isi1[i]} sedangkan file2: {isi2[i]}')

    if len1 > len2:
        for i in range (panjang, len1):
            count += 1
            print(f'baris ke-{i+1} berbeda, file1: {isi1[i]} sedangkan file2: -')

    elif len1 < len2:
        for i in range (panjang, len2):
            count += 1
            print(f'baris ke-{i+1} berbeda, file1: - sedangkan file2: {isi2[i]}')

    print(f"jumlah baris yang beda: {count}")

    handle1.close()
    handle2.close()
except:
    print("file tidak ditemukan")
```

Penjelasan:

Pada bagian awal program, pada baris pertama dan kedua, program akan meminta input dari user berupa dua nama file yang ingin dibandingkan lalu disimpan ke variabel **file1** dan **file2**. Setelah itu di baris ketiga sampai kelima, program akan menyiapkan dua list kosong yaitu **isi1** dan **isi2** untuk menampung isi masing-masing file, serta variabel **count** untuk menghitung jumlah perbedaan yang ada.

Lalu ada bagian **try**, pada bagian ini program akan mencoba membuka file menggunakan fungsi **open()** di baris delapan dan sembilan. Lalu pada baris sebelas sampai tiga belas, program melakukan looping untuk membaca setiap baris dari file pertama, membersihkan spasi pada awal atau akhir menggunakan **strip(**), lalu menyimpan ke dalam list **isi1**. Hal ini juga berlaku pada baris lima belas sampai tujuh belas untuk file kedua kemudian disimpan ke list **isi2**. Di baris sembilan belas dan dua puluh, program akan menghitung jumlah baris masing-masing file menggunakan **len()**. Kemudian di baris dua puluh dua sampai dua puluh tujuh, program akan menentukan panjang minimum dari kedua file dan menyimpannya ke variabel **panjang**. Ini dilakukan supaya saat proses perbandingan tidak terjadi error karena perbedaan jumlah baris. Dengan mengambil panjang yang lebih kecil, program bisa membandingkan bagian yang pasti ada di kedua file terlebih dahulu.

Selanjutnya di baris dua puluh sembilan sampai tiga puluh tiga, program melakukan perulangan untuk membandingkan isi kedua file berdasarkan index. Jika ditemukan perbedaan pada baris yang sama, maka **count** akan ditambahkan satu dan program akan mencetak informasi baris keberapa yang berbeda dan isi dari perbedaan pada masing-masing file. Pada baris tiga puluh lima sampai empat puluh dua, program akan menangani kasus dimana jumlah baris pada tiap file berbeda. Jika jumlah pertama lebih panjang, maka sisa barisnya dihitung sebagai perbedaan dan ditampilkan dengan keterangan file kedua tidak memiliki isi. Begitupun sebaliknya jika file kedua lebih panjang.

Pada baris empat puluh empat, program akan mencetak total jumlah baris yang berbeda menggunakan **count**. Setelah itu file akan ditutup menggunakan **close()** untuk menghindari

pemborosan resource. Jika terjadi error seperti file tidak ditemukan, maka bagian **except** akan dijalankan dan menampilkan pesan “file tidak ditemukan”

Output:

```
PS C:\Users\Acer\OneDrive\Dokumen\belajar-github\minggu_10> & C:\Users\Acer\AppData\Local\Programs\Python\Python313\python.exe "c:/Users/Acer/OneDrive/Dokumen/
belajar-github/minggu_10/soal 1.py"
masukkan nama file ke-1: soal1a.txt
masukkan nama file ke-2: soal1b.txt
baris ke-2 berbeda, file1: kapan bisa tidur sedangkan file2: mau mau
baris ke-3 berbeda, file1: saya mau makan sedangkan file2: kapan liburnya
baris ke-4 berbeda, file1: hidup pak karel sedangkan file2: pak karel
baris ke-5 berbeda, file1: bantai uas sedangkan file2: -
jumlah baris yang beda: 4
```

SOAL 2

Source code:

```
minggu_10 > soal2.py > ...
file1 = input("nama file1: ")
isi_soal = []
isi_jawaban = []
try:
    handle = open(file1)
    for line in handle:
            line = line.strip()
            line = line.split("||")
            soal = line[0].strip()
            jawaban = line[1].strip().lower()
            isi_soal.append(soal)
            isi_jawaban.append(jawaban)

    for i in range (len(isi_soal)):
        print(isi_soal[i])
        user_jawab = input("jawab: ")
        user_jawab = user_jawab.strip().lower()
        if user_jawab == isi_jawaban[i]:
            print("Jawaban benar!")
        else:
            print("Jawaban salah!")

    handle.close()

except:
    print("file tidak ditemukan")
```

Penjelasan:

Di bagian awal program, user diminta memasukkan nama file yang berisi soal dan jawaban lalu disimpan ke variabel **file1**. Setelah itu dibuat dua list kosong yaitu **isi_soal** untuk menampung pertanyaan dan **isi_jawaban** untuk menampung jawaban dari soal.

Kemudian pada bagian **try**, program akan membuka file menggunakan **open()** lalu membaca isinya baris per baris menggunakan perulangan. Setiap baris akan dibersihkan spasi berlebih pada awal dan akhir kalimat menggunakan **strip()** lalu akan dipisahkan menggunakan **split("||")** karena format soal dan jawaban dipisahkan menggunakan tanda **||**. Bagian soal akan disimpan ke **isi_soal** dan bagian jawaban akan disimpan ke **isi_jawaban** setelah diubah menjadi huruf kecil menggunakan **lower()** agar konsisten saat dibandingkan.

Selanjutnya, program akan melakukan looping sesuai dengan jumlah soal yang ada pada list i**si_soal**. Pada setiap iterasi, soal akan ditampilkan ke user menggunakan **print()** lalu meminta user memasukkan jawaban. Jawaban user kemudian dibersihkan menggunakan **strip()** dan diubah ke huruf kecil menggunakan **lower()**. Kemudian program akan membandingkan jawaban user dengan jawaban yang benar berdasarkan indeks yang sama. Jika sama maka akan dicetak "Jawaban benar!", sedangkan jika tidak sama akan muncul "Jawaban salah!". Setelah perulangan selesai, program akan menutup file menggunakan **close()** untuk memastikan tidak ada pemborosan resource. Jika terjadi kesalahan seperti file tidak ditemukan, maka program akan masuk ke bagian except dan mencetak "file tidak ditemukan".

Output:

```
nama file1: soal.txt
1+1 =
jawab: 3
Jawaban salah!
Bendera Indonesia?
jawab: merah putih
Jawaban benar!
Kota gudeg adalah:
jawab: jogja
Jawaban salah!
KomponenPC untuk penyimpanan file adalah...
jawab: harddisk
Jawaban benar!
50 *20 =
jawab: 1000
Jawaban benar!
```